DeskBuddy: Complete Project Guide & Placement Prep (v1 · v2 · C3)

Oct 10, 2026 · @Mugiwara_ya

1. The project in one minute

The problem. I kept picking up my phone just to check the time or one notification, and ten minutes later I was still scrolling. Switching the phone off wasn't an option either, because then I'd miss the calls and messages that matter.

The solution. DeskBuddy is a small desk device with a 0.96" OLED screen. It shows the clock and the phone notifications I choose, plus an animated face called **Mochi** that reacts to events. The phone stays face-down and out of reach.

30-second pitch (say this when asked "tell me about your project"):

> "DeskBuddy is an embedded desk companion I built in three versions. v1 was an ESP8266 that received phone notifications over MQTT and HTTP from an automation app. v2 moved to an ESP32 running FreeRTOS tasks, with my own Android app talking to it over a WebSocket. The latest version, DeskBuddy C3, is a pocket-sized build on an ESP32-C3 that connects to the same app over Bluetooth Low Energy or WiFi, with an optional battery and deep-sleep power switch. I also built a pip-installable command-line tool that downloads, flashes and monitors the firmware, with CI on GitHub Actions."

2-minute version. Add, in this order:

1. **Why three versions.** Each version fixed the weak points of the previous one:
   - v1's automation app was laggy, and its TLS setup skipped certificate checks.
   - v2 was powerful but desk-bound.
   - C3 is small, works without WiFi, and can run on a battery.
2. **One technical highlight.** In C3, only the Bluetooth stack runs in its own task. It hands complete messages to the main loop through a FreeRTOS queue, so there are no locks and no race conditions.
3. **One debugging story.** For example, the BLE-to-WiFi switch that crashed with a heap assert (section 10).

4. **What I'd do next.** Battery testing, a custom PCB, and making the app stop searching cleanly when it's closed.

Key numbers to remember

Fact	Value
Display	128×64 monochrome OLED, I2C, address 0x3C
Moods	15 moods, ~30 fps animation (C3)
Notifications kept	last 8, in a ring buffer
BLE	Nordic UART Service, MTU 247 (244 bytes of data per packet)
WiFi	WebSocket on port 81, HTTP on port 80, mDNS `deskbuddy-c3.local`
C3 flash layout	4 MB: two 1.875 MB app slots for OTA updates
Tool	`deskbuddy` CLI, 13 commands, 257 automated tests
Firmware tests	537 host-side checks of the pure logic

Being honest about AI help. I used Claude as a coding partner (Claude Code with the ECC plugin for the CLI tool). I set the idea and features, wired and tested every version on real hardware, found the bugs that only show up on a physical board, and made the design calls. If an interviewer asks, say exactly that. Then prove it by explaining any part of this document without notes. That is the goal of this guide.

2. Timeline: v1 → v2 → C3

	v1	v2	C3
Board	Wemos D1 Mini (ESP8266, 80 MHz, 1 core)	ESP32 WROOM DevKit (240 MHz, 2 cores)	ESP32-C3 Super Mini (RISC-V, 160 MHz, 1 core)
Display	SH1106 OLED 128×64, I2C	SSD1306 OLED 128×64, I2C	SSD1306 OLED 128×64, I2C

	v1	v2	C3
Code style	One 1,283-line `.ino` file, one big `loop()`	Modules + FreeRTOS tasks + event queue	Modules, one main loop + one queue from the BLE task
Build tool	Arduino IDE	PlatformIO (`espressif32@6.9.0`, Arduino core 2.0.17)	PlatformIO (pioarduino 55.03.36, Arduino core 3.3.6)
Phone link	Automation app (Automate/MacroDroid) → MQTT over TLS (HiveMQ) or HTTP GET	My own Android app → WebSocket over WiFi	Same app → **BLE** or WiFi WebSocket (switchable)
Settings	JSON file on LittleFS + web portal	One `Settings` struct in NVS (Preferences)	Same, saved only when something changed
Inputs	2 buttons	3 buttons + capacitive touch pad	2 buttons
Sound	Passive buzzer with `tone()` (blocking)	I2S DAC → PAM8403 speaker + optional buzzer	Passive buzzer with LEDC PWM (non-blocking)
Screens	Clock, weather, Mochi, notifications	+ pomodoro	Clock, Mochi, notifications
Power	USB	USB	USB, or Li-ion + TP4056 + deep-sleep power switch
Tooling	none	Python test client, PC face simulator	`deskbuddy` CLI + installer window, CI, releases

Why each change happened (the story to tell):

- **v1 → v2:**
  - The automation-app route was buggy and laggy, and needed a cloud MQTT broker.

  - The ESP8266 had too little RAM and only one core for animation, networking and sound at once.
  - So v2 moved to the dual-core ESP32 with FreeRTOS, and replaced MQTT with a direct WebSocket to my own app.
  - It also removed the API key that had been committed in the source.
- **v2 → C3:**
  - I wanted a pocket version: one small board, two buttons, no speaker or microphone.
  - The ESP32-C3 is tiny and cheap, and has Bluetooth LE, so it works where there's no WiFi.
  - I rewrote the firmware from scratch for the C3 and kept the app protocol, so the same Android app drives both versions.
- **C3 tooling:**
  - Flashing needed PlatformIO and about 1 GB of downloads.
  - So I made a `pip install`-able tool that downloads a verified release and flashes it with esptool. No IDE needed.

3. DeskBuddy v1: ESP8266 (Wemos D1 Mini)

Repo: `github.com/MONKEYDPARI019/DeskBuddy`. Firmware: `Firmware/DeskBuddy_v1.0_Stable.ino` (1,283 lines, Arduino IDE, board setting **NodeMCU 1.0 (ESP-12E Module)**).

3.1 Parts

Part	Job
Wemos D1 Mini (ESP8266)	Microcontroller with WiFi
SH1106 OLED 128×64, I2C	Display
3 LEDs (red, yellow, green) + 3 × 470 Ω	Status lights
2 push buttons	Mood/scroll, screen/silent/config
Passive piezo buzzer	Tunes and beeps
100 µF electrolytic + 0.1 µF ceramic across 3V3–GND	Decoupling. Added after random resets under WiFi + LED + buzzer load.

3.2 Pin-to-pin wiring and why

D1 Mini pin	GPIO	Connected to	Mode in code	Why this pin / concept
D2	GPIO4	OLED **SDA**	I2C data	Default I2C pins of the D1 Mini. `Wire.begin(D2, D1)`.
D1	GPIO5	OLED **SCL**	I2C clock	Bus at **400 kHz** (`Wire.setClock(400000)`) for faster redraws.
3V3 / GND	–	OLED VCC / GND	–	OLED logic is 3.3 V. The module has its own I2C pull-ups.
D5	GPIO14	BTN1 → GND	`INPUT_PULLUP`	Not a strapping pin, so a button can't block boot. **Active-low**: idle HIGH, pressed LOW.
D7	GPIO13	BTN2 → GND	`INPUT_PULLUP`	Same. BTN2 held at boot opens the config portal.
D6	GPIO12	470 Ω → red LED → GND	Output	Red = silent mode on.
D3	**GPIO0**	470 Ω → yellow LED → GND	Output	**Strapping pin**: must be HIGH at power-on, or the chip enters flash mode. Safe here only because nothing pulls it low before `setup()`.
D4	**GPIO2**	470 Ω → green LED → GND	Output	**Strapping pin**: must be HIGH at boot. The D1 Mini's onboard LED is also on it. Green = WiFi/MQTT state.
D8	**GPIO15**	Buzzer + (− to GND)	`tone()`	**Strapping pin**: must be LOW at boot. A piezo doesn't pull it high, so it boots fine.
A0	ADC	nothing	`analogRead(0)` once	Only to seed the random generator for Mochi's random eye movement.

Interview point: v1 used three of the ESP8266's strapping pins (0, 2, 15) for outputs. It worked, but it was fragile: a pull resistor on any of them would stop the board booting. In v2 I left all strapping pins free on purpose.

3.3 How the v1 code works

`setup()`:

1. Serial at 115200. Buttons as `INPUT_PULLUP`. LEDs and buzzer as outputs.
2. Start I2C at 400 kHz and the OLED (`U8G2_SH1106_128X64_NONAME_F_HW_I2C`, full frame buffer).
3. `loadConfig()` reads `/config.json` from **LittleFS** into a `Config` struct: OpenWeatherMap key/city/units, MQTT broker/port/user/pass/topic, HTTP on/port/path, silent flag.
4. **BTN2 held for 0.5 s?** Then start the config portal: soft AP **`DeskBuddy_Config`**, web page at **192.168.4.1** with `/save` and `/resetwifi`. Stop here.
5. Otherwise, **WiFiManager** `autoConnect("DeskBuddy_Setup")`. It opens a captive portal if no WiFi is saved.
6. `configTime(19800, 0, "pool.ntp.org")`: NTP time with a +5:30 offset (19800 s = 5.5 h).
7. MQTT client (`PubSubClient` on `WiFiClientSecure`, port **8883**, TLS) and the HTTP server with `/notify`.
8. Fetch weather, play the intro tune.

`loop()`, every 50 ms (`FRAME_MS`, about 20 fps):

1. Handle HTTP clients, then `handleButtons()` and `updateLEDs()`.
2. If WiFi is up:
   - MQTT reconnect (at most every 5 s) or `mqttClient.loop()`;
   - weather refresh every 10 minutes.
3. Clear the frame buffer, then draw the current screen (clock / weather / Mochi / notifications).
4. Draw the popup overlay, `sendBuffer()`, then `delay(50)`.

Notifications. MQTT and HTTP both call one shared function, `pushNotification(app, title, body)`:

- It writes into a **circular buffer** of 8 (`notifHead = (notifHead + 1) % MAX_NOTIFS`).

- It counts unread, shows a popup and plays a chime.
- MQTT payloads are JSON `{"app","title","body"}`, parsed with ArduinoJson 6 (`StaticJsonDocument<512>`).
- HTTP is `GET /notify?title=..&msg=..`.

Mochi. Two `Eye` structs with position, size and pupil targets.

- `updatePhysics()` moves each value a fraction of the way to its target every frame (`cur + (tgt − cur) × k`, k = 0.12–0.14). This is **exponential smoothing**, which is why the motion looks soft.
- It adds random blinks (every 1.8–5 s, 130 ms long) and random glances ("saccades") every 0.5–2 s.
- A sine-wave breathing offset (`sin(t/900) × 2`) moves the eyes up and down.
- 15 moods, each drawn with U8g2 primitives (discs, boxes, lines).

Buttons:

- BTN1 press: next mood, or next notification on the notification screen.
- BTN2 short: next screen. BTN2 held 2 s or more: silent mode, saved to flash.

LEDs:

- Red = silent.
- Yellow = unread notifications.
- Green: solid = WiFi + MQTT, blinking = WiFi only, off = no WiFi.

3.4 v1 weaknesses (good to admit in interviews)

- **Blocking sound.** `pTone()` calls `delay()`. The boot tune and the mood tunes froze the whole loop, including the display and MQTT, while they played.
- **TLS without verification.** `espClient.setInsecure()` encrypts the MQTT link but doesn't check the server's certificate, so a man-in-the-middle is possible.
- **Secrets in the code/repo.** The weather API key was in source history.
- **Laggy notification path.** Phone → automation app → cloud broker (HiveMQ) → ESP8266. That's many hops, and the automation app was unreliable.
- **One giant file and `delay()`-based timing.** Hard to extend.

Bugs I fixed in v1 (from the changelog):

- `MOOD_COUNT` was 13 while 15 moods existed, so two moods were never shown.

- `drawMouth()` was never called, so Mochi had no mouth.
- The config portal function existed but was never called from `setup()`.
- PubSubClient's default **256-byte buffer** was too small for the TLS packets. Fixed with `setBufferSize(1024)`.
- The melodies were written but never wired to events.

4. DeskBuddy v2: ESP32 + FreeRTOS + Android app

Repo: `github.com/MONKEYDPARI019/DeskBuddy-v2`. PlatformIO project: `platform = espressif32@6.9.0` (Arduino-ESP32 core **2.0.17**), board `esp32dev`. Firmware version `2.0.0-dev`, app version `0.2.0`.

4.1 What changed from v1 and why

Change	Reason
ESP8266 → **ESP32 WROOM** (dual core, 520 KB SRAM, DAC, I2S, touch)	Room for animation + networking + audio at the same time
Arduino IDE → **PlatformIO**	Pinned library versions, repeatable builds, OTA env, exception decoder
Single `.ino` → **modules** (`main`, `config`, `events`, `display`, `face`, `sensors`, `audio`, `mic`, `net`, `cli`)	Each file has one job; easier to test and explain
One `loop()` → **7 FreeRTOS tasks** + one **event queue**	Sound and networking no longer freeze the display
MQTT/HiveMQ + automation app → **WebSocket server** on the ESP32 + **own Android app**	Lower latency, no cloud, no third-party app
LittleFS JSON + web portal → **NVS** `Settings` struct + serial CLI + app	Settings are typed, small and atomic; WiFi-only portal via WiFiManager
API key in source → set at runtime (`set owm_key`)	Never commit secrets
SH1106 → **SSD1306**	The OLED I had (different controller, see section 6)

4.2 Pin-to-pin wiring and why (30-pin ESP32 DevKit)

GPIO	Connected to	Why this pin / concept
21	OLED SDA	The ESP32's default I2C pins. Bus at 400 kHz, OLED at 0x3C.
22	OLED SCL	Same.
32	BTN1 → GND	`INPUT_PULLUP`, active-low. GPIO 32/33 are safe general pins (not strapping, not flash).
33	BTN2 → GND	Same.
27	BTN3 → GND	Same. Hold = push-to-talk.
4	Touch pad (copper tape/coin)	**T0 capacitive touch channel**. `touchRead()` drops when a finger adds capacitance. Threshold `touch_th` is set by calibration.
16 / 17 / 18	Red / yellow / green LED, each with a resistor to GND	Plain outputs, chosen away from strapping pins.
25	PAM8403 L input through 10–22 kΩ	**DAC1**: the built-in 8-bit DAC, driven by I2S0 in "built-in DAC" mode. A larger series resistor fixed the hiss.
26	(DAC2, right channel)	Enabled together with DAC1 (`I2S_DAC_CHANNEL_BOTH_EN`).
19	Passive buzzer through 100–470 Ω	**LEDC** PWM channel, for the optional emotion sounds.
14	INMP441 SCK	I2S1 bit clock (BCLK).
13	INMP441 WS	I2S1 word select (left/right clock).
34	INMP441 SD	I2S1 data in. **34–39 are input-only**, which is fine for data coming in.
VIN (5 V)	PAM8403 +5V	The amplifier needs 5 V. OLED and mic use 3V3.

Pins I avoided on purpose (common interview question):

- **6–11:** wired to the SPI flash chip. Using them crashes the board.

- **0, 2, 5, 12, 15:** strapping pins. GPIO12 HIGH at boot selects 1.8 V flash and the board won't boot.
- **34–39:** input-only, with no internal pull-ups, so they can't be used for buttons without external resistors.
- **ADC2 pins:** can't do analog reads while WiFi is on. I only used them digitally, for touch and the DAC.

4.3 How the v2 code works

Tasks (created with `xTaskCreatePinnedToCore`):

Task	Core	Priority	Stack	Job
audio	0	4	4 KB	Plays chirps; streams speech PCM to the DAC
app	1	3	6 KB	**Only consumer of the event queue**: all behaviour
input	1	3	3 KB	Buttons (OneButton), touch, LEDs at 50 Hz
mic	0	3	4 KB	INMP441 capture while push-to-talk is held
net	0	2	8 KB	WiFi, NTP, mDNS, OTA, HTTP, WebSocket, weather
ui	1	2	6 KB	Draws face + screens at about 30 fps
cli	1	1	4 KB	Serial commands

Rules that kept it stable:

1. **One event queue** (32 slots of `Event {type, value, text[48]}`).
   - Producers only `postEvent()`.
   - Only the app task reads the queue.
   - Event types: `EVT_BUTTON`, `EVT_TOUCH`, `EVT_NOTIFICATION`, `EVT_FACE_CMD`, `EVT_SCREEN_CMD`, `EVT_NET_STATUS`, `EVT_APP_LINK`, `EVT_SPEAK_START`, `EVT_OTA`, and more.
2. **One recursive mutex** ("UI lock") protects face and screen data shared between the app task and the ui task.
3. **Lock order:** never send on the network while holding the UI lock. This prevents deadlock.
4. **Audio calls only enqueue work** (`xQueueSend` to a 12-slot audio queue). Speech goes through a **16 KB stream buffer** that absorbs WiFi jitter.

5. **Networking runs on core 0, next to the WiFi stack; the UI runs on core 1.** Heavy network work can't make the animation stutter.

Phone link.

- The ESP32 is the **WebSocket server** (`ws://deskbuddy.local:81/`) and advertises **mDNS** `_deskbuddy._tcp` .
- Why the device is the server: phones change IP and Android kills background servers, but a phone can always open an *outgoing* connection. And several clients (phone + laptop) can connect at once.
- Messages are JSON text frames. Speech and mic audio are binary frames of 16-bit PCM.
- HTTP `GET /notify?app=..&title=..&msg=..` is kept for curl/Tasker.

Other pieces:

- WiFiManager portal **`DeskBuddy_Setup`** .
- NTP clock, weather every 10 minutes.
- ArduinoOTA (`pio run -e ota -t upload` → `deskbuddy.local`).
- Sleep after `sleep_sec` (5 min) dims the OLED.
- Custom partition table: two 1.75 MB OTA slots (the firmware was about 1.1 MB).

Face (v2 onwards).

- Each mood is a set of **parameters** (eye size, lid angles, pupil position, mouth), not a hand-drawn bitmap.
- So the face can glide between moods.
- On top of the moods, there is a **state machine**: boot → idle ⇄ notify / listening → thinking → speaking / sleep.
- I checked every face on a PC with a small **face simulator** (`tools/face-sim`), which mocks `Arduino.h` and `U8g2lib.h` and renders a PNG sheet.

Tools I used in v2:

- VS Code + PlatformIO, and `pio device monitor` with `esp32_exception_decoder` and `send_on_enter` .
- `tools/deskbuddy_client.py` , a Python WebSocket test client.
- Android Studio for the app.

Status:

- Working on hardware: OLED, faces, screens, buttons, touch, LEDs, WiFi portal, weather, app + notification forwarding, speaker and buzzer.
- Not finished: the INMP441 mic (ordered) and OTA (not yet verified).

5. DeskBuddy C3: the pocket version

Repo: `github.com/MONKEYDPARI019/DeskBuddy-C3` (release **v2.0.0**, firmware `c3-2.0.0`). Folder layout: `firmware/` (PlatformIO), `app/` (Android), `tool/` (Python CLI), `.github/workflows/` (CI).

5.1 What changed from v2 and why

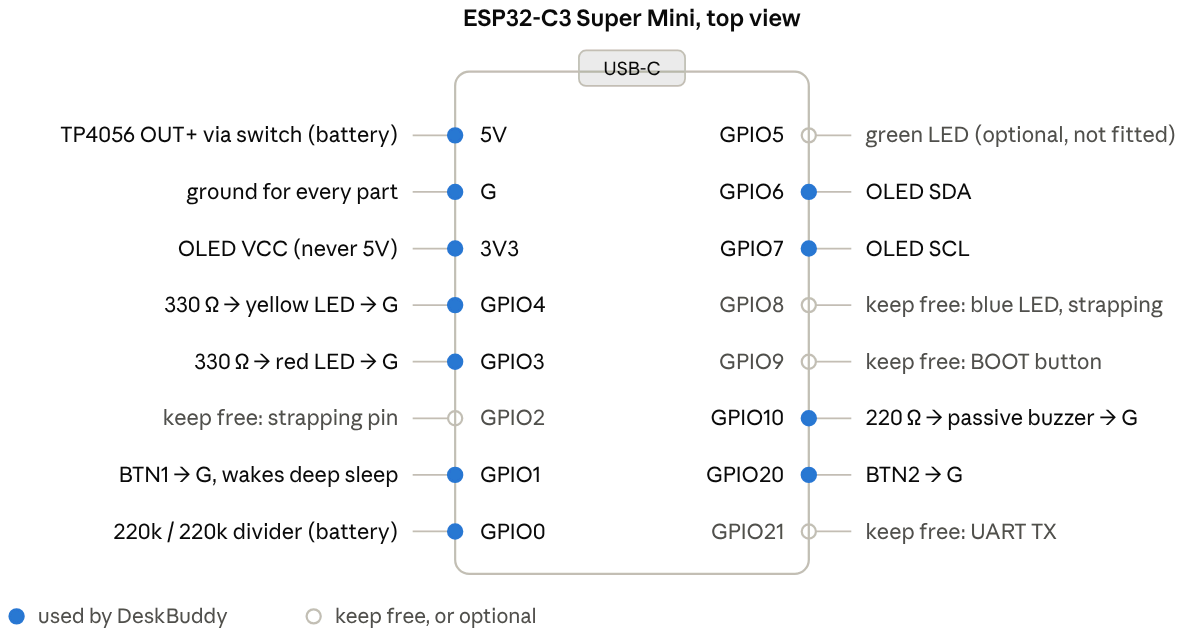
Change	Reason
ESP32 → **ESP32-C3 Super Mini** (RISC-V, 1 core, 400 KB SRAM, BLE 5 + WiFi, native USB)	Thumb-sized, cheap, has BLE, can run on a battery
WiFi only → **BLE (default) or WiFi**, switchable	BLE works anywhere with no router and uses less power
7 tasks + mutex → **one main loop + one queue** from the BLE task	One core anyway, so fewer moving parts and no locks
3 buttons + touch + speaker + mic → **2 buttons + buzzer**	Pocket size; the app does the rest
Screens: clock, weather, Mochi, notifications, pomodoro → **clock, Mochi, notifications**	Weather needs WiFi and an API key; kept it simple
Arduino core 2.0.17 → **pioarduino / core 3.3.6**	Core 2.0.x could not drive I2C reliably on this board (the OLED stayed black)
New: **battery build** (TP4056 + Li-ion, deep-sleep power switch, battery icon, auto-off)	Portable use
New: **`deskbuddy` CLI + installer UI**, CI, GitHub Releases	Flash without an IDE; repeatable releases
App protocol **kept**	The same Android app drives v2 and C3 (device type chosen in Setup)

5.2 Pin-to-pin wiring and why

Super Mini pin	Connected to	Why this pin / concept
3V3	OLED VCC	OLED logic is 3.3 V. **Not 5V.**
G	OLED GND, button legs, LED cathodes, buzzer −	One shared ground for every return current.
GPIO6	OLED SDA	Any C3 GPIO can be I2C (the GPIO matrix). 6/7 are free and next to each other.
GPIO7	OLED SCL	Probed at 100 kHz; U8g2 draws at 400 kHz.
GPIO1	BTN1 leg; diagonal leg → G	`INPUT_PULLUP`, active-low. **Must be GPIO 0–5**: only those can wake the C3 from deep sleep (power button in the battery build).
GPIO20	BTN2 leg; diagonal leg → G	Normally UART0 RX. Free here because our serial runs over **native USB**.
GPIO3	330 Ω → **red LED** → G	Silent mode; fast blink = low battery.
GPIO4	330 Ω → **yellow LED** → G	Unread notifications.
GPIO5	330 Ω → green LED → G (**optional, I left it out**)	App link. The firmware works without it.
GPIO10	220 Ω → passive buzzer + ; buzzer − → G	LEDC PWM output. The resistor limits the GPIO current.
GPIO0	Battery divider midpoint (battery build only)	ADC1 channel 0. Never read in the USB build, so a floating pin can't switch the board off.
GPIO2, 8, 9	**left free**	**Strapping pins.** 8 = onboard blue LED, 9 = BOOT button.
GPIO18 / 19	(USB D− / D+)	Native USB-Serial/JTAG. Not on the header.

Super Mini pin	Connected to	Why this pin / concept
GPIO21	left free	UART0 TX (boot messages).

DeskBuddy C3 wiring: strapping pins 2, 8 and 9 stay free



ESP32-C3 Super Mini header · top view, USB-C up

The pin order follows the usual Super Mini layout. Check your board's silkscreen, because some clones differ.

LED resistor maths (interview favourite):

- A red LED drops about 2.0 V, so the resistor sees 3.3 − 2.0 = 1.3 V.
- I = 1.3 V ÷ 330 Ω ≈ **4 mA**. That is bright enough, and far below the GPIO limit (about 20 mA recommended, 40 mA absolute max).
- v1 used 470 Ω → ≈ 2.8 mA.

Buttons: diagonal legs.

- A 4-leg tactile switch is two pairs of legs that are always joined internally. Pressing connects the pairs.
- Wiring two legs of the same pair = the button always reads "pressed". That gave me phantom presses: Mochi changed face on its own.
- Diagonal legs are always in different pairs, so they always work.

5.3 Battery build (env `c3-battery`, `-DFEATURE_BATTERY=1`)

From	To	Why
Li-ion + / − (my cell: 3.8 V, 1500 mAh, from a hotspot)	TP4056 **B+ / B−**	The TP4056 charges the cell (CC/CV up to 4.2 V) and its protection chip cuts off over-discharge and short circuits.
TP4056 **OUT+**	slide switch → Super Mini **5V**	The 5V pin feeds the board's **LDO regulator**, which makes a clean 3.3 V.
TP4056 **OUT−**	Super Mini **G**	Common ground.
TP4056 B+	220 kΩ → **GPIO0** → 220 kΩ → G	**Voltage divider** halves the battery voltage so the ADC can read it.

Rules:

- **Never feed the battery into 3V3.** A full cell is 4.2 V, and the chip's maximum is 3.6 V.
- About 3.7 V into the 5V pin works: the LDO needs only a small headroom, so the board runs down to about 3.5 V.
- Switch the battery off before plugging into a PC. On many Super Minis, USB 5 V is tied straight to the 5V pin.
- Charge through the TP4056's own USB port.
- Use only the + and − contacts of a 3-contact phone/hotspot battery.

Divider maths:

- 4.2 V ÷ 2 = **2.1 V** at GPIO0. That fits the ADC range at 11 dB attenuation (about 0–2.5 V on the C3).
- Current: 4.2 V ÷ 440 kΩ ≈ **9.5 µA**. That is tiny, so the divider can stay connected.
- Code: 8 readings of `analogReadMilliVolts()` averaged, × 2, plus a calibration offset (`set bat_off`).

Behaviour:

- Readings are trusted only after they look like a real cell (3.0–4.5 V) for a while.
- ≤ 10 %: red LED fast blink and a warning.
- Below **3.35 V for 30 s**: "Bye!", then deep sleep.

- **Hold BTN1 3 s** to power off and on. A shorter press (a bump in the pocket) goes straight back to sleep.

Rough runtime on 1500 mAh: BLE with the screen on 25–35 h; WiFi 12–18 h (estimates, not yet measured).

5.4 Controls

Input	Click	Hold
BTN1	Next mood (on the inbox screen: next notification)	⩾ 0.8 s: silent mode. Battery build: 3 s = power off
BTN2	Next screen: clock → Mochi → inbox	Clear notifications
Both held while pressing RESET	Switch Bluetooth ↔ WiFi (saved to flash)	

After `sleep_sec` (120 s) without input, Mochi falls asleep and the screen dims. **The first press only wakes him.** I once thought this was a "BTN1 hang".

6. Every concept used, explained

For each concept: what it is, where DeskBuddy uses it, and the line an interviewer wants to hear.

6.1 Electronics

GPIO, pull-ups and active-low buttons.

- A GPIO input left unconnected "floats" and reads random values.
- `INPUT_PULLUP` connects an internal resistor (about 45 kΩ on ESP32) to 3.3 V, so the pin reads HIGH when idle. The button connects the pin to GND, so pressed = LOW (**active-low**).
- Why not active-high? It needs an external pull-down resistor per button, and a GND-switched button is safer when wires touch.

Switch bounce and debouncing.

- A mechanical contact bounces for a few milliseconds and produces many edges.

- C3 accepts a new state only after it has been **stable for 30 ms**. Then it measures how long the button was held: ≥ 0.8 s = long press, ≥ 3 s = power (battery build).
- v1 used `delay(150)` after a press, which is crude and blocks the loop.
- v2 used the OneButton library.

Current-limiting resistors.

- Ohm's law: R = (Vsupply − Vled) ÷ I.
- 330 Ω → about 4 mA per LED. 220 Ω in series with the buzzer limits the GPIO current.

Strapping pins. Pins the chip reads at reset to decide the boot mode.

Chip	Strapping pins	Rule
ESP8266 (v1)	GPIO0, 2, 15	0 and 2 HIGH, 15 LOW for normal boot
ESP32 (v2)	GPIO0, 2, 5, 12, 15	12 HIGH = 1.8 V flash, won't boot
ESP32-C3 (C3)	GPIO2, 8, 9	9 LOW = download mode (the BOOT button)

Voltage divider.

- Vout = Vin × R2 ÷ (R1 + R2). Two equal resistors halve the voltage.
- High values (220 kΩ) waste only about 10 µA.
- The ADC's input resistance and the small sampling capacitor make very high values a bit noisy, so the code averages 8 readings.

Decoupling capacitors.

- WiFi transmit bursts pull hundreds of mA for microseconds, and the 3.3 V rail dips.
- A big electrolytic (100 µF) supplies the burst. A small ceramic (0.1 µF) filters high-frequency noise.
- Without them, v1 had **brown-out resets** (the chip resets when its supply drops too low).

LDO regulator.

- Turns 5 V (or a 3.5–4.2 V battery) into a steady 3.3 V by dropping the extra voltage as heat.
- Needs a small "dropout" headroom. That's why the battery goes to the 5V pin (into the regulator), never to 3V3.

Li-ion cell and TP4056.

- Cell voltages: 4.2 V full, about 3.7 V nominal, about 3.0 V empty.
- The TP4056 charges with **constant current, then constant voltage** (CC/CV) up to 4.2 V.
- The 6-pad module adds a protection IC (over-discharge, over-current, short circuit).
- A "3.8 V" high-voltage cell charged to 4.2 V is safe; it just isn't filled to its maximum.

Passive vs active buzzer.

- **Active:** has its own oscillator; apply DC, get one fixed beep.
- **Passive:** a bare piezo; you must drive a square wave at the note's frequency. Only passive can play tunes.

PWM (LEDC on ESP32).

- A square wave with a set frequency and duty cycle.
- For the buzzer, frequency = pitch and duty = loudness (50 % is the loudest).
- Arduino core 3.x changed the API from channels (`ledcSetup` + `ledcAttachPin`) to pins (`ledcAttach(pin, freq, bits)`). I had to handle this when moving C3 to core 3.

ADC (analog to digital converter).

- ESP32 ADCs are 12-bit SAR converters.
- **Attenuation** sets the input range: 11 dB ≈ 0–2.5 V on the C3.
- The raw values are non-linear. `analogReadMilliVolts()` uses the chip's factory calibration (eFuse) to return millivolts.
- On the classic ESP32, **ADC2** can't be used while WiFi is on.

DAC and I2S (v2 only).

- The classic ESP32 has two 8-bit DACs (GPIO25/26). v2 drove them through I2S0 in "built-in DAC" mode to play PCM audio into a PAM8403 class-D amplifier.
- **I2S** is a 3-wire digital audio bus: bit clock (BCLK), word select (left/right), data. The INMP441 MEMS microphone outputs I2S.

Capacitive touch (v2).

- The pin measures how long a tiny capacitor takes to charge. A finger adds capacitance, so `touchRead()` goes **down**.
- Threshold = halfway between the untouched and touched values.

6.2 Inside the microcontroller

Flash, partitions and OTA.

- Flash holds the bootloader, a **partition table** (at 0x8000) and the partitions.
- C3 layout (4 MB):
  - `nvs` 0x9000 (20 KB), settings and WiFi credentials;
  - `otadata` 0xE000, which says which app slot to boot;
  - `app0` 0x10000 and `app1` 0x1F0000, 1.875 MB each;
  - `spiffs` 128 KB;
  - `coredump` 64 KB.
- **OTA:**
  1. The new firmware is written into the *inactive* slot while the old one keeps running.
  2. If it is complete and valid, otadata switches to the new slot and the board reboots.
  3. A failed upload never bricks the board.
- The bootloader sits at **0x0 on the C3** (and at 0x1000 on the classic ESP32). This matters for the merged release image, which is flashed at 0x0.

NVS (non-volatile storage).

- A key-value store in flash with wear levelling. Arduino's `Preferences` wraps it.
- C3 keeps settings in namespace **`dbc3`** (`tz`, `link`, `silent`, `bright`, `vol`, `sleep`, `wsport`, …).
- **It writes only when a value actually changed:** a copy of what's in flash is compared with `memcmp`. Flash cells survive roughly 10k–100k erase cycles.

Deep sleep and wake.

- Deep sleep turns off the CPU and most RAM. Only the RTC domain stays on (micro-amps).
- Waking is a **reset**: `setup()` runs again, and `esp_sleep_get_wakeup_cause()` tells you why.
- On the C3, only **GPIO 0–5** can wake from deep sleep. That's why BTN1 is on GPIO1.
- `gpio_hold_en()` keeps the pull-up active while asleep.

Native USB-Serial/JTAG (C3).

- The C3 has USB built into the chip (USB ID **303A:1001**), so there's no CH340/CP2102 converter chip.

- Toggling DTR/RTS resets it. So the monitor opens the port with DTR/RTS off (`monitor_dtr = 0`, `monitor_rts = 0`).
- `Serial.setTxTimeoutMs(0)` stops printing from blocking when no PC is reading.

Download (bootloader) mode.

- Hold **BOOT** (GPIO9 LOW), tap **RESET**, release BOOT. The ROM bootloader then waits for esptool.
- Needed when the running firmware has crashed the USB.

6.3 Buses and wired protocols

I2C.

- Two wires: SDA (data) and SCL (clock). Both are **open-drain**, so they need pull-up resistors (usually on the OLED module).
- A master addresses a slave by a 7-bit address; the slave answers with an **ACK**.
- Speeds: 100 kHz standard, 400 kHz fast.
- The OLED is at **0x3C** (some at 0x3D). U8g2 wants the 8-bit form, so the code calls `setI2CAddress(0x3C × 2 = 0x78)`.
- C3 probes at 100 kHz (works even with weak pull-ups), and U8g2 switches to 400 kHz for drawing.
- `Wire.setTimeOut(10)`: a stuck bus can't freeze the loop.

SSD1306 vs SH1106.

- Both are 128×64 OLED controllers. The SH1106 has 132-column RAM, so the image is offset by 2 columns, and it lacks some addressing modes.
- So the wrong driver shows a shifted or garbled picture. v1 used `U8G2_SH1106_...`; v2/C3 use SSD1306.

Frame buffer (U8g2 `_F_`).

- 128 × 64 pixels ÷ 8 = **1024 bytes** of RAM.
- Every frame: `clearBuffer()` → draw everything → `sendBuffer()` sends the whole image over I2C. No flicker, because the screen only ever gets complete frames.

UART vs USB CDC.

- **UART:** two wires (TX/RX) at an agreed baud rate (115200). v1 and v2 reach the PC through a USB-UART converter chip.
- The C3 uses native USB, where the baud rate doesn't really matter.

6.4 Wireless and networking

WiFi modes.

- **STA (station):** joins your router.
- **AP (access point):** the board creates its own network. Used for the setup portal at 192.168.4.1.

Captive portal (WiFiManager).

- In AP mode, a small DNS server answers *every* name with 192.168.4.1. So the phone's "sign in to network" page opens the setup page.
- You pick your WiFi and type the password. The credentials are stored in NVS, and the board reboots into STA mode.
- C3 portal name: `DeskBuddy_C3_Setup`. It times out after 180 s and retries the saved network.

TX power and modem sleep (C3 WiFi fixes).

- The Super Mini's tiny antenna distorts at full power, so phones couldn't see its setup network. `WiFi.setTxPower(WIFI_POWER_8_5dBm)` fixed it.
- **Modem sleep** turns the radio off between router beacons to save power, but then multicast (mDNS) packets get missed. So `WiFi.setSleep(false)`.

IP, TCP, ports.

- IP finds the device; TCP gives a reliable ordered byte stream; the port picks the service.
- HTTP on **80**, WebSocket on **81**, OTA (espota) on **3232**.

HTTP.

- A request/response protocol. `GET /notify?title=Hi&msg=Hello` puts parameters in the URL query string.
- It's stateless: every notification is a new connection. Fine for curl and automation apps, wasteful for a live link.

WebSocket.

- Starts as an HTTP request with `Upgrade: websocket`, then stays open as a **full-duplex** TCP connection carrying text or binary frames.
- Both sides can push at any time, with no polling. Ping/pong keeps it alive and detects dead links (the app pings every 10 s).

MQTT (v1).

- **Publish/subscribe** through a **broker**. The phone publishes to topic `deskbuddy/notify`, the ESP8266 subscribes, and the broker (HiveMQ Cloud) forwards.
- Ports: 1883 plain, **8883 over TLS**.
- Good for many devices behind NAT, but it needs the internet and a cloud account. That added latency.

TLS.

- Encrypts the connection *and* proves the server's identity with a certificate chain.
- v1's `setInsecure()` kept the encryption but skipped the identity check. That is vulnerable to man-in-the-middle.
- The proper fix: pin the broker's CA certificate (`setTrustAnchors` / `setCACert`).

mDNS / DNS-SD.

- Name lookup without a DNS server: multicast to 224.0.0.251, port 5353.
- The C3 answers **`deskbuddy-c3.local`** and announces the service **`_deskbuddy._tcp`** (port 81) with TXT record `model=c3`.
- The Android app finds it with **NSD** (Network Service Discovery). That's the FIND button.

NTP and time zones.

- **WiFi mode:** NTP gets UTC time from internet servers.
- **BLE mode:** there's no internet, so the app sends `{"type":"time","epoch":..,"tz_min":330}` on every connect.
- Epoch = seconds since 1 Jan 1970 UTC.
- **POSIX TZ strings have the sign inverted:** `"UTC-5:30"` means India, which is UTC **+**5:30. Classic gotcha.
- v1 used `configTime(19800, 0, ...)`: 19800 s = 5.5 h.

Bluetooth Low Energy.

- **GAP** (who talks):
  - The peripheral (DeskBuddy) **advertises**; the central (phone) **scans** and **connects**.
  - The name `DeskBuddy-C3` goes in the **scan response**, because the 31-byte advertising packet is full with the 128-bit service UUID.
- **GATT** (what they say): a server holds **services**, which hold **characteristics**, each with a UUID.

- DeskBuddy uses the **Nordic UART Service** (NUS):

Role	UUID	Property
Service	`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`	–
RX (app writes)	`…02`	write, write-without-response
TX (device sends)	`…03`	**notify**

- **Notifications:** the phone enables them by writing to the **CCCD** descriptor (0x2902) of TX.
- **MTU:**
  - Default 23 bytes (20 bytes of data).
  - The app requests **247**, giving 244 bytes per packet (MTU − 3-byte ATT header).
  - A JSON message can still be longer, so each message ends with `\n`. The receiver joins packets until the newline (`LineAssembler`), and the sender splits long messages (`frameChunks`).
- BLE TX power is set to 3 dBm: enough for a desk, and it saves battery.
- **Library: NimBLE-Arduino 2.x**, a lighter BLE stack than Bluedroid, with less RAM and flash.

6.5 Software and RTOS

Super loop vs RTOS.

- v1: one `loop()`. Anything slow (a `delay()` in a tune, a blocking HTTP call) froze everything.
- v2: FreeRTOS tasks.
- C3: one main loop again, but **non-blocking**: every module does a little work and returns.

Non-blocking timing with `millis()`.

- Instead of `delay(500)`, check `if (millis() - last >= 500) { last = millis(); ... }`.
- The subtraction is **overflow-safe**: `millis()` wraps after 49.7 days, and unsigned subtraction still gives the right difference.

FreeRTOS basics:

- **Task:** an independent function with its own stack and a priority. The **scheduler** is preemptive, so a higher-priority ready task interrupts a lower one.
- **Queue:** a thread-safe FIFO that *copies* items. The safest way to pass data between tasks.
- **Mutex:** one task at a time in a critical section. It has priority inheritance against **priority inversion**. A **recursive mutex** can be taken again by the task that already holds it.
- **Semaphore:** a signal or counter. `...FromISR()` variants are the only calls allowed inside interrupts.
- **Pinned to core:** `xTaskCreatePinnedToCore` (v2 put networking on core 0 with the WiFi stack).

C3 concurrency model.

- NimBLE runs its host in its own task.
- Its `onWrite` callback only feeds bytes into `LineAssembler`, and each **complete line** is copied into a 4-slot FreeRTOS queue (`inbox`).
- The main loop takes lines out with `xQueueReceive(..., 0)` (no waiting) and handles them.
- Nothing else is shared between tasks, so there are no locks, no deadlocks and no races.

Event-driven design / state machines.

- The face has states (boot, idle, notify, sleep…) with timed transitions.
- The buttons are a small state machine (up → down → held → released).
- The CLI's serial monitor is a reconnecting state machine.

Ring buffer.

- A fixed array of 8 notifications. New ones overwrite the oldest: `head = (head + 1) % 8`.
- No dynamic memory, so no fragmentation on a long-running device.

Animation by interpolation.

- Each eye parameter moves toward its target by a fraction every frame (`x += (target − x) × k`), which looks smooth and organic.
- C3 hides mood swaps behind a quick blink.

JSON and ArduinoJson.

- Every message is one JSON object with a `"type"` field.
- v1/v2 used ArduinoJson 6 (`StaticJsonDocument<N>` , fixed size). C3 uses **ArduinoJson 7** (`JsonDocument` , grows as needed).

UTF-8 to ASCII.

- The OLED font only has ASCII.
- Notifications in emoji or Hindi arrive as multi-byte UTF-8, so `text.h` folds accented letters (é → e), drops what it can't show, and word-wraps to the screen width.
- That's also why one line from the app can be up to 1536 bytes.

Conditional compilation (feature flags).

- `#if FEATURE_BATTERY` compiles the battery code in or out.
- The USB build never reads GPIO0, so a floating pin can't trigger a false "battery empty" shutdown.

Host unit testing.

- Pure logic (framing, text, notification store, time zones, battery curve) lives in `src/logic/` with no Arduino dependencies.
- So it compiles with g++ on a PC: 537 checks, run in CI.

7. How the C3 firmware works

About 3,300 lines of C++ in `firmware/src/` . All pins and timings live in one file, `config.h` .

7.1 The modules

File	Job, in one line
`main.cpp`	`setup()` boot order and the main `loop()`
`config.h`	Pins, timings, `FW_VERSION` , `FEATURE_BATTERY`
`settings.*`	Load/save the `Settings` struct in NVS (`dbc3`), only when changed

File	Job, in one line
`oled.*`	Find the OLED (0x3C/0x3D) at 100 kHz with retries; re-check every 3 s; re-init after a blip
`face.*`	Mochi: 15 moods + states, parametric eyes, blinks, gaze
`ui.*`	Draws the clock, Mochi and inbox screens, popups and overlays (reads state, changes nothing)
`app.*`	Behaviour: screens, notifications, sleep, LEDs, pushing state to the app, link switching, power-off
`protocol.*`	Parses incoming JSON and builds outgoing JSON (ArduinoJson 7)
`link.*`, `link_ble.cpp`, `link_wifi.cpp`	One interface (`linkBegin/Send/Receive/Service`) over two radios + the inbox queue
`buttons.*`	30 ms debounce, click / long / power events, simulated presses for the CLI
`leds.*`	Patterns: on, off, fast blink, heartbeat
`buzzer.*`	Non-blocking melodies on LEDC; volume = duty cycle
`battery.*`, `power.*`	ADC reading, low/empty logic, deep-sleep power switch (battery build only)
`cli.*`	Serial commands (`help`, `status`, `set`, `test`, `link`, …)
`logic/*.h`	Pure C++ with no Arduino: `framing.h`, `text.h`, `notif_store.h`, `tz.h`, `battery_curve.h`. Unit-tested on a PC

7.2 Boot sequence (`setup()`)

1. **`powerWakeCheck()`** (battery build): if woken by BTN1, keep checking that it's held for 3 s; if released early, go straight back to deep sleep.

2. `Serial.begin(115200)` and `setTxTimeoutMs(0)`. Wait up to 1 s for a USB monitor, then print the banner `=== DeskBuddy C3 c3-2.0.0 (USB build) ===`.
3. `settingsLoad()` → `ledsBegin()` → `buttonsBegin()` → `buzzerBegin()` → `batteryBegin()`.
4. `oledBegin()`: wait until 500 ms after power-on (OLED start-up time), then probe up to 10 times. Show the splash screen.
5. **Both buttons held?** Flip `settings.link` between BLE and WiFi, save, wait for release. This switches the radio without a PC.
6. `appBegin()` (face, brightness, time zone) → `linkBegin(settings.link)` (start BLE *or* WiFi) → play the hello sound.

7.3 The main loop

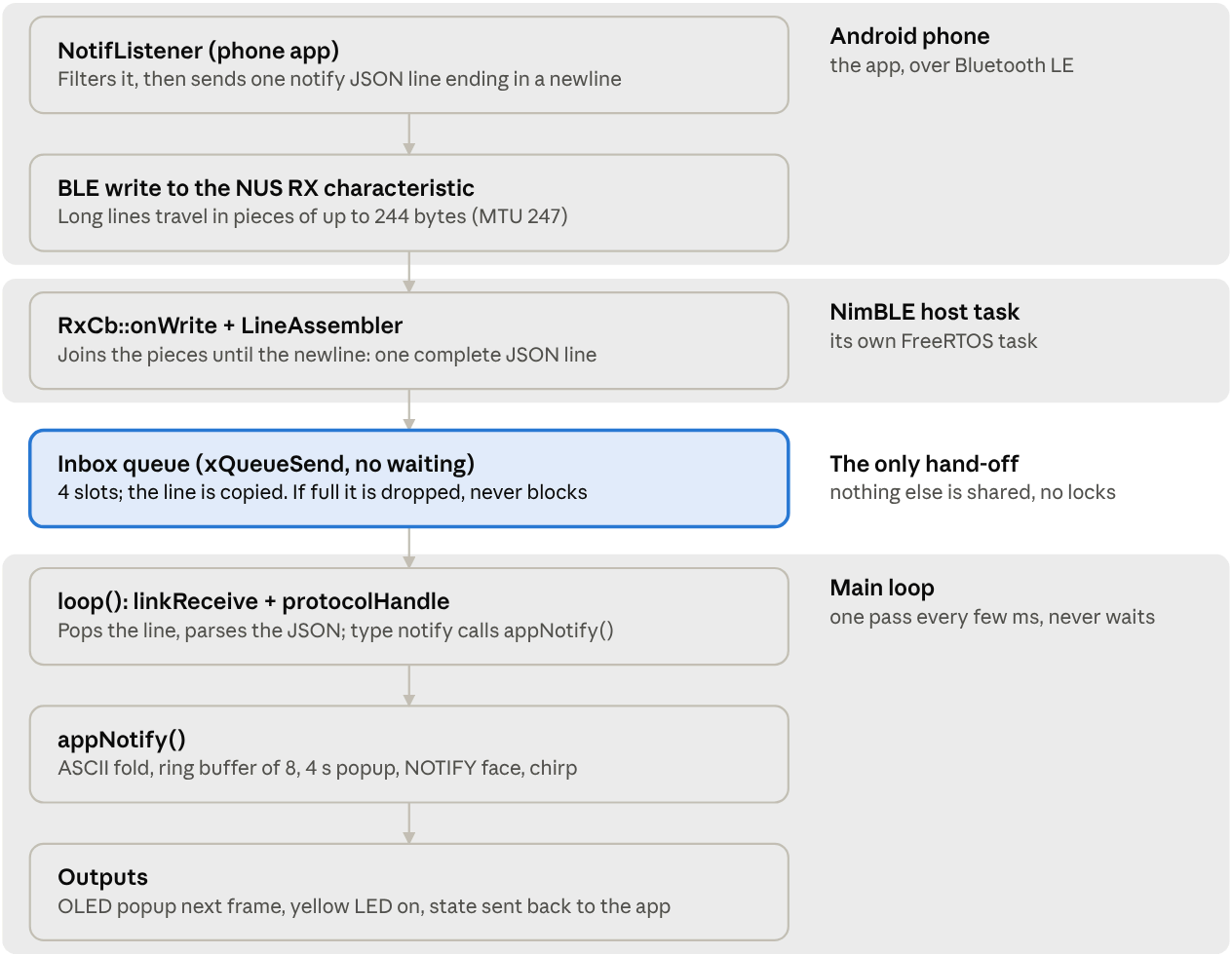
```
void loop() {
    cliService();                    // 1. serial commands
    linkService();                   // 2. radio housekeeping
(WiFi/WebSocket/OTA)
    while (linkReceive(msg, sizeof(msg)))
        protocolHandle(msg, replyToApp);   // 3. every complete message from
the app
    ButtonPress p = buttonsPoll();    // 4. buttons
    if (p.ev != BtnEvent::None) appButton(p);
    batteryUpdate();                 // 5. battery (no-op in USB build)
    appLoop();                       // 6. timers: sleep, popups, state
push, restart
    buzzerUpdate();                  // 7. next note if it's time
    oledService();                   // 8. OLED health check every 3 s
    if (millis() - lastFrame >= frame) {   // 9. draw at ~30 fps (10 fps
asleep)
        lastFrame = millis();
        faceUpdate(); uiRender();
    }
    delay(1);                        // let the idle task feed the watchdog
}
```

Why this order? Inputs first (serial, radio, buttons), then logic, then outputs (sound, LEDs, display). Every step returns quickly, so one pass takes a few milliseconds, and nothing ever waits.

Why `delay(1)`? On ESP32, the FreeRTOS idle task must run now and then, or the **task watchdog** fires. A 1 ms delay yields the CPU.

7.4 A notification's journey (be able to draw this)

A notification crosses from the BLE task to the main loop through one queue



C3 notification path · 7 steps, 2 tasks, 1 queue

Practise drawing this on a whiteboard. The highlighted queue is the answer to "how do you avoid race conditions?".

1. **Phone.**
   - Android calls `NotifListener.onNotificationPosted()`.
   - The app skips ongoing notifications, group summaries, duplicates within 3 s and blocked apps.

- It sends `{"type":"notify","app":"WhatsApp","title":"Mom","body":"Dinner is ready"}` followed by `\n`.

2. **BLE transport.**
   - The app writes the bytes to the NUS **RX** characteristic, in pieces of at most 244 bytes.
3. **NimBLE task.**
   - `RxCb::onWrite()` pushes the bytes into `LineAssembler`.
   - On `\n`, the complete line is copied into the **inbox queue** (`xQueueSend`, no waiting). If the queue is full, the line is counted as dropped instead of blocking the BLE stack.
4. **Main loop.**
   - `linkReceive()` pops the line.
   - `protocolHandle()` parses the JSON, sees `type == "notify"`, and calls `appNotify()`.
5. **`appNotify()`:**
   - folds UTF-8 to ASCII (app 15 chars, title 31, body 127);
   - wakes Mochi;
   - pushes the notification into the 8-slot ring buffer (`notif_store.h`);
   - shows a 4 s popup (unless the inbox is already on screen);
   - sets the face to the NOTIFY state;
   - plays the notify chirp;
   - marks the state "dirty".
6. **Outputs.**
   - The yellow LED turns on (unread > 0).
   - The next frame draws the popup.
   - `appLoop()` sees the dirty state and sends `state` back to the app, so the app's live Mochi copy updates.

Over WiFi it's the same from step 4. The only difference: the line arrives as a **WebSocket text frame** (or an HTTP `GET /notify`) instead of BLE writes.

7.5 Switching radio (BLE ↔ WiFi)

Three ways:

1. **The app** sends `{"type":"set","key":"link","value":"wifi"}`.

2. **Serial:** `link wifi` or `deskbuddy link wifi`.
3. **Hardware:** both buttons held while pressing RESET.

What the firmware does (`appSwitchLink`):

1. Saves the new mode to NVS.
2. Replies `{"type":"link_switch","link":"wifi","restart_ms":1500}`, so the app knows to follow.
3. Shows "Switching to WiFi" and **restarts after 1.5 s**.

Why restart instead of switching live? BLE and WiFi share one radio and a lot of RAM. A clean boot into one mode is simpler and more reliable than tearing a stack down and starting another. That teardown was exactly where the heap crash happened (section 10).

7.6 WiFi mode in detail (`link_wifi.cpp`)

1. Set the hostname, set the mode, then **TX power 8.5 dBm** after every mode change, and modem sleep off.
2. If credentials are saved, connect (15 s timeout). If not, or if that fails, open the WiFiManager portal `DeskBuddy_C3_Setup` (180 s, then retry).
3. Once connected:
   - **mDNS** `deskbuddy-c3` with service `_deskbuddy._tcp` + TXT `model=c3`, `fw=...`;
   - **ArduinoOTA** (progress bar on the OLED);
   - **WebSocket server** on `wsPort` (81);
   - **HTTP server** on `httpPort` (80) with `/notify`.
4. NTP sync for the clock.

7.7 BLE mode in detail (`link_ble.cpp`)

1. `NimBLEDevice::init("DeskBuddy-C3")`, `setMTU(247)`, `setPower(3)`.
2. Create the server with `setCallbacks(&serverCb, false)`. **`false` = NimBLE must not delete my static callback object.** This was the crash fix.
3. Create the NUS service: RX (write / write-no-response) and TX (notify).
4. Advertise the service UUID, with the name in the scan response.
5. When the phone disconnects, advertise again.
6. **Sending:** `frameChunks()` splits each JSON line into MTU-sized pieces and notifies each one.

7. **Stopping:** stop advertising and disconnect. No full `deinit`.

7.8 Sleep, screen saver and LEDs (`app.cpp`)

Sleep:

- No input for `sleep_sec` (120 s) → `enterSleep()`: go to the Mochi screen, set the SLEEP face, contrast 1, draw at 10 fps.
- Any button, notification or app command → `wake()`. **The press that wakes him does nothing else.**

LEDs:

- Red: fast blink if the battery is low, else solid if silent, else off.
- Yellow: on if there are unread notifications.
- Green (optional): solid = app connected, fast blink = setup portal, heartbeat = waiting.
- All LEDs are off while Mochi sleeps.

State push: when something changed ("dirty"), and every 15 s, the device sends the full `state` JSON (face, mood, screen, unread, silent, volume, link, rssi, battery, uptime…). That's how the app mirrors the device.

7.9 The OLED hot-plug logic (`oled.cpp`)

- **At boot:** wait for 500 ms of uptime, then probe 0x3C/0x3D at 100 kHz up to 10 times (about 1 s). If nothing answers, print a bus scan with a wiring hint, and keep going without a screen.
- **Every 3 s:** probe again.
  - Missing and now answering → `begin()` it.
  - It missed and came back → **re-initialise it**, because an SSD1306 that lost power for a moment stays dark until it is initialised again.
- This fixed my "OLED not found / screen blank" problem on the Super Mini, together with moving to Arduino core 3.x.

7.10 Serial CLI (115200, or `deskbuddy monitor`)

Command	Does
`help`, `status`, `get`, `state`	Info: status summary, all settings, the JSON state

Command	Does
`set <key> <value>`, `defaults`	Change settings (`silent`, `brightness`, `volume`, `sleep_sec`, `tz`, `leds`, `bat_off`, …)
`link ble` / `link wifi`	Switch radio (restarts)
`screen clock|mochi|notify|next`, `face <mood>`, `sound <name>`, `notify <text>`	Drive the device by hand
`press 1|2 [ms]`	Simulate a button press (hold it for `ms` to make a long press)
`time <epoch>`	Set the clock manually
`i2c`, `oled reinit`	Scan the I2C bus, restart the OLED
`test oled|leds|buzzer|buttons|battery`	Hardware checks
`wifi forget`, `reboot`, `off`	Forget WiFi, restart, power off (battery build)

8. The Android app

Kotlin + **Jetpack Compose**, package `com.deskbuddy.app`, minSdk 26 (Android 8.0), targetSdk 34, OkHttp 4.12 for WebSockets. About 2,600 lines. Retro "DeskBuddy OS" look (Press Start 2P and VT323 fonts). One app drives **v2 and C3**: you pick the device type in Setup.

8.1 Files and what they do

File	Job
`MainActivity.kt`	Entry point: asks for permissions, starts/stops the link, hosts the Compose UI
`NotifListener.kt`	A **NotificationListenerService**: Android calls it for every notification posted on the phone
`DeskBuddyClient.kt`	A Kotlin `object` (singleton): the one connection to DeskBuddy, over a WebSocket *or* BLE, same JSON either way

File	Job
`BleLink.kt`	BLE central: scan with a filter on the NUS service UUID → connect GATT → request MTU 247 → discover services → enable TX notifications (CCCD 0x2902) → send lines
`LinkService.kt`	A **foreground service** (type `connectedDevice`) with an ongoing notification, so Android doesn't kill the connection in the background
`Discovery.kt`	**NSD** (mDNS) search for `_deskbuddy._tcp`; the name hint "c3" picks the C3 and ignores a v2 on the same WiFi
`Prefs.kt`	SharedPreferences: device type, link, host/port, blocked apps, theme…
`AppState.kt`	Observable state for the UI (last `state` from the device, log, known apps)
`ui/Screens.kt`, `ui/Components.kt`, `ui/Mochi.kt`, `ui/Theme.kt`	Home (live Mochi copy), Moods, Inbox, Setup; Mochi redrawn on the phone from the `state` messages
`VoiceLoop.kt`, `TtsPcm.kt`	v2 only: push-to-talk echo and text-to-speech to the speaker (hidden for C3)

8.2 How forwarding works on the phone

1. The user grants **Notification access**. Android then binds `NotifListener`.
2. `onNotificationPosted(sbn)` checks, in order:
   - forwarding is on;
   - the notification isn't from DeskBuddy itself;
   - it isn't ongoing (music, navigation) or a group summary;
   - it has a title;
   - it isn't a duplicate (same key and text within 3 s; messaging apps re-post);
   - the app isn't blocked in Inbox;
   - the client is connected.

3. Then `DeskBuddyClient.notify(app, title, text)` sends the `notify` JSON over whichever transport is active.

8.3 Connection logic

On connect, the app sends `hello`, then (for a C3) `time` with `epoch` + `tz_min` (330 for India). In BLE mode the board has no internet, so this is its only clock source.

Reconnect with exponential backoff:

- BLE: 2 s, 4 s, 8 s … up to 30 s.
- WebSocket: starts at 1 s.
- Backoff stops a lost device from draining the phone's battery with constant retries.

WebSocket keep-alive: OkHttp sends a ping every 10 s, so a dead WiFi link is noticed quickly.

Link switch: when the device replies `link_switch`, the app changes transport and follows the board after its restart.

Why a foreground service? Since Android 8, background apps are limited. A foreground service with a visible notification may keep running. Android 14 requires a declared type (`connectedDevice`) and the matching permission.

8.4 Permissions (from the manifest)

Permission	Why
`INTERNET`, `ACCESS_NETWORK_STATE`	WebSocket over WiFi
`CHANGE_WIFI_MULTICAST_STATE`	Receive mDNS multicast for FIND
`BLUETOOTH_SCAN`, `BLUETOOTH_CONNECT` (Android 12+)	BLE scanning and connecting
`BLUETOOTH`, `BLUETOOTH_ADMIN`, `ACCESS_FINE_LOCATION` (Android ≤ 11)	Older phones needed location permission to scan BLE
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_CONNECTED_DEVICE`	The background link service
`POST_NOTIFICATIONS` (Android 13+)	Show the service's notification

Permission	Why
Notification access (special setting, not a manifest permission)	Read other apps' notifications

Don't pair DeskBuddy in Android's Bluetooth settings. The app connects directly over GATT. Bonding isn't needed, and a stale bond can block connecting.

8.5 Known issue (fix written, not applied yet)

The problem. When the app is closed, the foreground notification keeps searching for the board, and the user has to stop it from inside the app.

The prepared fix (`app-notification-fix.zip`, version 0.3.0):

- a **Stop** action on the notification;
- searching gives up after 10 minutes and shows a swipe-away "DeskBuddy not found" notification with **Search again**;
- an `onTaskRemoved()` handler;
- a setting **"Stay connected when app is closed"**.

In an interview, this is a good example of a real UX trade-off: background reliability versus battery and user control.

9. Tools and commands, version by version

9.1 Tools I used

Tool	Used for	Version(s)
Arduino IDE (ESP8266 core)	Write, compile and upload v1	v1
VS Code + **PlatformIO**	Build, upload, serial monitor, library pinning, OTA env	v2, C3
Android Studio (Gradle, Kotlin)	Build and install the app	v2, C3
Git + GitHub	Version control, repos for each version	all

Tool	Used for	Version(s)
GitHub Actions	CI (build both firmware envs, host tests, tool tests on Ubuntu + Windows) and releases	C3
Python 3 + pip	The `deskbuddy` tool, the v2 test client	v2, C3
esptool	Talks to the ESP ROM bootloader: flash, erase, read the chip	C3 tool (PlatformIO uses it too)
pyserial	Serial ports from Python	C3 tool
pytest, ruff, mypy	Tests, linting/formatting, type checking	C3 tool
g++ host tests	Test the pure logic on a PC	C3
Claude (chat) and Claude Code with the **ECC** plugin	Coding partner; Claude Code built the CLI repo end to end from my master prompt	v2, C3
HiveMQ Cloud, MacroDroid/Automate	MQTT broker and phone automation	v1
OpenWeatherMap API	Weather	v1, v2
curl	Send test notifications over HTTP	v1, v2, C3
Multimeter	Find button leg pairs, check the battery voltage	C3
KiCad 10	PCB design (in progress)	v2

9.2 v1 commands (Arduino IDE)

1. Boards Manager: install the **ESP8266** core. Library Manager: install `U8g2`, `PubSubClient`, `WiFiManager` (tzapu) and `ArduinoJson`.
2. **Tools → Board → NodeMCU 1.0 (ESP-12E Module)**, pick the COM port, then **Upload**.
3. Open the Serial Monitor at **115200**.
4. Test a notification:

```
curl "http://<device-ip>/notify?title=Mother&msg=Dinner+is+ready"
```

5. Config mode: hold BTN2 while resetting. Join `DeskBuddy_Config` and open `http://192.168.4.1`.

9.3 v2 commands (PlatformIO)

```
pio run                      # build (env esp32dev)
pio run -t upload            # build + flash over USB
pio device monitor           # serial CLI at 115200 (exception decoder on)
pio run -e ota -t upload     # flash over WiFi to deskbuddy.local
```

Serial CLI examples:

```
set owm_key <key>
set owm_city Bengaluru
save
status
test oled | leds | buzzer | speaker | buttons | touch | mic
face love
screen weather
notify hello
wifi reset
reboot
```

PC test client:

```
pip install websockets
python tools/deskbuddy_client.py monitor
python tools/deskbuddy_client.py face love
python tools/deskbuddy_client.py notify "WhatsApp" "Mom" "Dinner is ready"
curl "http://deskbuddy.local/notify?app=Gmail&title=New%20mail&msg=Hello"
```

9.4 C3 commands

PlatformIO (open the `firmware` folder):

```
pio run -e c3 -t upload              # USB build
pio run -e c3-battery -t upload      # battery build (-DFEATURE_BATTERY=1)
pio device monitor                   # 115200, DTR/RTS off
pio run -e ota -t upload             # over WiFi to deskbuddy-c3.local
```

The `deskbuddy` tool (PowerShell or a Linux shell):

```
pip install "git+https://github.com/MONKEYDPARI019/DeskBuddy-
C3#subdirectory=tool"
deskbuddy doctor                 # 7 checks: Python, esptool, pyserial,
board, port free, GitHub, permissions
deskbuddy ports                  # * = ESP32-C3 (USB ID 303A)
deskbuddy versions               # releases on GitHub
deskbuddy flash                  # latest release, SHA-256 checked, keeps
settings
deskbuddy flash --env battery    # battery build
deskbuddy flash --erase          # wipe settings + WiFi too
deskbuddy flash --local          # build from source with a private
PlatformIO, then flash
deskbuddy monitor --log run.txt  # reconnecting serial monitor
deskbuddy status
deskbuddy cmd i2c
deskbuddy link wifi
deskbuddy wifi-forget
deskbuddy reboot
deskbuddy build --env usb --out dist
deskbuddy ui                     # Tkinter installer window
```

Raw esptool (what the tool does underneath; esptool 5 spells commands with dashes):

```
python -m esptool --chip esp32c3 -p COM9 -b 921600 write-flash 0x0 deskbuddy-
c3-c3-v2.0.0.bin
python -m esptool --chip esp32c3 -p COM9 erase-flash
```

Releases:

```
# bump FW_VERSION in firmware/src/config.h and add a CHANGELOG section first
git tag v2.1.0
git push origin v2.1.0           # release.yml builds, checksums and
publishes
```

9.5 How the `deskbuddy` tool works (if asked)

Packaging. A Python package (`deskbuddy-cli` 2.0.0) with a console-script entry point `deskbuddy`. It depends only on `esptool` and `pyserial`.

`flash`:

1. Asks the GitHub Releases API for the release.
2. Downloads the **merged image** (bootloader + partition table + boot_app0 + app in one `.bin` for address 0x0).
3. Checks its **SHA-256** against `SHA256SUMS`, and caches it in `~/.deskbuddy/firmware`.
4. Runs `python -m esptool` as a subprocess, parsing the progress. It tries 921600 baud first, then falls back to 460800 and 115200.
5. Unless `--erase` is given, writes the image in two pieces that **skip NVS (0x9000–0xE000)**, so the WiFi and settings survive the update.
6. If the chip won't sync, prints "Hold BOOT, tap RESET, release BOOT" and retries up to 3 times.

`monitor`. Opens the port with DTR/RTS off. A small state machine reconnects after resets and unplugging. Errors are coloured.

`build`. Uses its own **private PlatformIO** in a venv (on Windows in `C:\.deskbuddy\platformio`, overridable with `DESKBUDDY_PIO_CORE`), so it never breaks your normal setup.

Exit codes (useful for scripts):

Code	Meaning
0	ok
2	usage error
3	no board
4	port busy
5	network
6	checksum
7	flash failed
8	no reply
9	build failed
130	Ctrl+C

`ui`. A Tkinter window: board, firmware and build pickers, an INSTALL button with a progress bar, and the serial monitor with quick buttons. It's a thin layer over the same `installer.py`, `serial_io.py` and `monitor.py` as the CLI.

Quality. 257 pytest tests (fakes for serial, subprocess and HTTP, so no board is needed), about 94 % coverage. ruff and mypy run in CI on Ubuntu and Windows, Python 3.9 and 3.12.

10. Debugging stories (STAR format)

Interviewers love "tell me about a hard bug". Each story is **S**ituation, **T**ask, **A**ction, **R**esult. Learn two or three well; the rest are one-liners.

Story 1: the BLE → WiFi switch crashed the board

- **Situation:** On the C3, switching from Bluetooth to WiFi rebooted the board with `assert failed: heap_caps_free` in the serial log.
- **Task:** Make radio switching reliable, because the app depends on it.
- **Action:**
  1. The assert came from the heap allocator: `free()` was called on memory that wasn't allocated on the heap.
  2. Only the switch path freed things, and that pointed to the BLE teardown.
  3. In **NimBLE-Arduino 2.x**, `setCallbacks(cb)` defaults to `deleteCallbacks = true`. On deinit, NimBLE calls `delete` on my callback object, which was a **static** object, not one created with `new`.
  4. I changed the call to `setCallbacks(&serverCb, false)`.
  5. I also redesigned the switch: save the new mode to NVS, tell the app (`link_switch`), and **restart** into the new radio instead of tearing one stack down at runtime.
- **Result:** Switching works every time. Lesson: read the ownership rules of a library's API, and prefer a clean restart over complex runtime teardown on small devices.

Story 2: the OLED stayed black on the Super Mini

- **Situation:** The same OLED worked in a simple test sketch, but my firmware printed `[OLED] NOT FOUND` and the screen stayed blank.
- **Task:** Get the display working reliably, including after loose-wire blips.

- **Action:**
  1. Isolated the problem with a minimal I2C test.
  2. Found two causes:
     - the old Arduino core (2.0.x) had I2C problems on this board;
     - probing once at 400 kHz right after power-on was too early for the OLED.
  3. Moved to **pioarduino (Arduino core 3.3.6)**.
  4. Probe at **100 kHz**, wait for 500 ms of uptime, and retry 10 times.
  5. Add a re-check every 3 s that **re-initialises** the SSD1306 after a power blip, and `Wire.setTimeOut(10)` so a stuck bus can't freeze the loop.
- **Result:** The OLED comes up every boot and recovers on its own if a wire wiggles.

Story 3: phones couldn't see the WiFi setup network

- **Situation:** In WiFi mode, the board said the portal `DeskBuddy_C3_Setup` was open, but my phone didn't list it.
- **Task:** Make first-time WiFi setup work.
- **Action:**
  1. Ruled out the firmware logic: the serial log showed the AP started.
  2. Researched the board: the ESP32-C3 Super Mini's tiny antenna is known to distort the signal at full TX power.
  3. Set `WiFi.setTxPower(WIFI_POWER_8_5dBm)` after **every** WiFi mode change, because the mode change resets it.
  4. Turned modem sleep off, so mDNS answers reliably.
- **Result:** The setup network shows up, phones connect at 192.168.4.1, and FIND works. Lesson: some bugs are RF and hardware, not code.

Story 4: "BTN1 hangs the device"

- **Situation:** Pressing BTN1 seemed to do nothing, and sometimes Mochi changed face by himself.
- **Task:** Find out whether it was code or hardware.
- **Action:**
  1. Used the serial log and `test buttons`.
  2. Found two separate things:

    - Mochi was **asleep**, and by design the first press only wakes him, so it looked like a hang;
    - the phantom presses came from wiring two legs of the **same internal pair** of the tactile switch.
  3. Wired the diagonal legs, which are always different pairs. Checked with the multimeter's beep mode.
- **Result:** The buttons behave. Lesson: reproduce with logs before changing code, and know the hardware.

Story 5: the board always booted in WiFi mode

- **Situation:** After flashing the new firmware, it started in WiFi mode every time, even though the default is BLE.
- **Action:**
  1. Settings live in **NVS**, which flashing the app doesn't erase. An old `link=wifi` value from earlier firmware was still there.
  2. Fixed it with `link ble`.
  3. Added the **both-buttons-at-reset** toggle, so it can be fixed without a PC.
  4. Later made the flashing tool deliberately keep or erase NVS (`--erase`).
- **Result:** Predictable mode, plus a hardware escape hatch.

Story 6: updating the firmware wiped the saved WiFi

- **Situation:** Flashing the merged release image (at 0x0) erased the WiFi credentials every time.
- **Action:** The merged image covers the whole start of the flash, including the NVS area (0x9000–0xE000), with padding. The tool now writes the image in two pieces around NVS, unless the user asks for `--erase`.
- **Result:** Updates keep settings, and a covering test makes sure it stays that way.

More one-liners

Version	Problem	Fix
v1	MQTT messages cut off	PubSubClient's 256-byte default buffer → `setBufferSize(1024)`

Version	Problem	Fix
v1	Random resets under load	Brown-out → 100 µF + 0.1 µF decoupling
v1	Two moods never shown	`MOOD_COUNT` was 13, not 15
v2	Speaker hiss	Larger series resistor into the PAM8403
v2	API key leaked in git history	Removed it, set it at runtime, rotate the key
C3	Compile errors after a library update	NimBLE 1.x → 2.x changed callback signatures; pinned NimBLE 2.5.1
C3	Battery build could switch itself off	Floating GPIO0 read as "empty" → read it only in the battery build, and only after plausible readings
C3	Clock showed `--:--` in BLE mode	No internet; the app now sends `time` on every connect
C3	PlatformIO build "hung"	It was a silent first-time download (~1 GB) / a corrupt Python package; reinstalled it, and the tool now uses its own private PlatformIO

11. Design decisions and trade-offs

"Why did you do it this way?" is the most common follow-up. Each answer names the alternative I rejected.

Decision	Why	Alternative and its cost
ESP32-C3 for the pocket build	Small, cheap, BLE + WiFi, native USB, low power	Classic ESP32: bigger, more power. nRF52: no WiFi
BLE as default link	Works anywhere with no router, lower power, no setup	WiFi only: needs a network and a portal
Nordic UART Service	A widely known "serial over BLE" layout; easy to test with nRF Connect	Custom GATT per feature: more work, no benefit for JSON lines

Decision	Why	Alternative and its cost
JSON lines + `\n` framing	Human-readable, same messages over BLE and WebSocket, easy to debug	Binary protocol: smaller, but harder to debug and extend
Device = WebSocket server (v2/C3)	Phones change IP and can't host reliable servers; many clients can connect	Device as client: needs to know the phone's address
Own Android app instead of an automation app	Reliable, low latency, per-app filter, no cloud	Automate/MacroDroid + MQTT: laggy, cloud-dependent (v1)
Single loop + one queue (C3)	One core; simplest correct design; no locks	Many tasks + mutexes (v2): needed for audio streaming on two cores
Restart to switch radio	Clean state, no teardown bugs, low RAM	Live switching: complex; crashed in testing
NVS, save only on change	Typed key-value, atomic, wear-levelled; fewer flash writes	LittleFS JSON file (v1): parsing, more code, can corrupt on power loss mid-write
Pinned library versions	Same build everywhere; no surprise API changes	Floating versions: NimBLE 1→2 broke the build
PlatformIO	Version pinning, multiple envs (USB, battery, OTA), CLI for CI	Arduino IDE: manual, hard to reproduce
`FEATURE_BATTERY` compile flag	USB build can never misread a floating ADC pin	Runtime setting: risk of wrong config switching the device off
Passive buzzer on LEDC	Real tunes, volume by duty cycle, non-blocking	`tone()` + `delay()` (v1): froze the loop
U8g2 full frame buffer	Flicker-free whole frames, simple drawing	Page mode: less RAM, more complex redraws
WiFi TX power 8.5 dBm	Super Mini antenna distortion fix	Full power: setup network invisible

Decision	Why	Alternative and its cost
Merged image + SHA-256 + skip NVS	One file to flash at 0x0, verified, keeps settings	Separate bins: easy to flash at wrong offsets
Foreground service in the app	Android keeps the link alive in the background	Background service: killed by Android 8+

Limitations I know about (say them before the interviewer does)

- **No link security.**
  - The BLE link is unauthenticated: anyone nearby with the app could send notifications. The WebSocket has no token either.
  - Fix: BLE bonding with a passkey shown on the OLED, and an auth token on the WebSocket.
- **Battery estimate from voltage only.** Li-ion voltage vs charge isn't linear and sags under load. A fuel-gauge IC (e.g. MAX17048) would be accurate.
- **Battery runtime not measured yet.** The 25–35 h figure is an estimate.
- **Notification privacy.** Message text is sent in plain text over the local link.
- **App not in CI**, because there is no Gradle wrapper yet.
- **Still a perfboard.** The PCB is in progress (KiCad).

What I'd do next

1. Apply the app fix (Stop action, give-up timer).
2. Test the battery build and measure the current draw.
3. Add BLE pairing.
4. Design the PCB with USB-C and the charger on board.
5. Make an enclosure.

12. Interview question bank

Cover the answer column and say your answer out loud first. If you can answer these, you can handle most follow-ups.

12.1 About the project

Question	Answer
What does DeskBuddy do?	Shows the clock and filtered phone notifications on a small OLED, with an animated face, so the phone can stay away while studying.
What was your role?	Idea, feature set, wiring and testing on hardware, debugging the physical problems, design decisions. I used Claude as a coding partner and can explain every part.
Why three versions?	v1 proved the idea but was laggy and fragile. v2 rebuilt it properly on ESP32 with my own app. C3 made it pocket-sized with BLE and a battery option.
What's the hardest part you solved?	The BLE → WiFi crash: a NimBLE 2.x callback-ownership issue causing `heap_caps_free` asserts (section 10, story 1).
What would you improve?	BLE pairing/security, a fuel-gauge IC, measured battery life, a PCB and enclosure, the app's background behaviour.
How long does a notification take to appear?	Effectively instant over BLE (well under a second; one GATT write plus parsing and drawing). v1's cloud MQTT path was noticeably slower. *(Not formally measured.)*
How did you test it?	On hardware with serial `test` commands; 537 host unit checks of the pure logic; 257 pytest tests for the tool; CI on every push.
Is it a product?	A working prototype. Next steps toward a product are a PCB, an enclosure, security, and power measurements.

12.2 Hardware and electronics

Question	Answer
Why `INPUT_PULLUP` for buttons?	It gives a defined HIGH when idle without external resistors. The button pulls the pin to GND, so pressed = LOW (active-low).
What is debouncing and how did you do it?	Contacts bounce for a few ms. I accept a new state only after it's stable for 30 ms, then time the press for click / long / power.
How did you pick the LED resistor?	R = (3.3 − Vf) / I. With about 2 V forward voltage and 330 Ω that's about 4 mA: visible, and safe for the GPIO.
What are strapping pins? Which did you avoid?	Pins read at reset to choose the boot mode. C3: GPIO2, 8, 9. ESP32: 0, 2, 5, 12, 15. ESP8266: 0, 2, 15 (v1 used them, which was fragile).
Why is BTN1 on GPIO1?	Only GPIO 0–5 can wake the ESP32-C3 from deep sleep, and BTN1 is the power button in the battery build.
Why not connect the battery to 3V3?	A full Li-ion is 4.2 V and the chip's maximum is 3.6 V. The 5V pin goes through the onboard LDO, which regulates to 3.3 V.
How do you measure the battery voltage?	A 220 kΩ + 220 kΩ divider halves it to ⩽ 2.1 V, within the ADC's 11 dB range. Average 8 `analogReadMilliVolts()` readings, × 2.
Why such large divider resistors?	To keep the current tiny (about 9.5 µA), so the divider doesn't drain the battery. The cost is a bit more noise, so I average.
What does the TP4056 do?	Charges a single Li-ion cell with constant current, then constant voltage, to 4.2 V. The module's protection IC cuts off over-discharge and short circuits.
Passive vs active buzzer?	Active has a built-in oscillator (one tone with DC). Passive needs a square wave at the note frequency, so only passive can play melodies.

Question	Answer
What is PWM and where did you use it?	A square wave with a variable duty cycle. I used LEDC PWM for the buzzer: frequency = pitch, duty = volume.
Why decoupling capacitors?	WiFi transmit bursts cause supply dips. Bulk 100 µF + 0.1 µF ceramic near the supply prevented v1's brown-out resets.
SSD1306 vs SH1106?	Both are 128×64 OLED controllers. The SH1106 has 132-column RAM (a 2-pixel offset), so each needs its own driver.
What's on the I2C bus and at what address?	Only the OLED, at 0x3C (some at 0x3D). U8g2 takes the 8-bit address 0x78.
Why are I2C lines pulled up?	They're open-drain: devices only pull low, and the resistors pull the line high. The OLED module has them on board.
What did you use the DAC and I2S for? (v2)	I2S0 drove the ESP32's built-in DAC to play PCM speech into a PAM8403 amplifier. I2S1 read the INMP441 microphone.
Why is GPIO34 OK for the mic data?	34–39 are input-only. Mic data is an input, so that's fine (but they have no pull-ups for buttons).
How does capacitive touch work?	The pin measures charge time. A finger adds capacitance, so `touchRead()` drops below a calibrated threshold.

12.3 Firmware and RTOS

Question	Answer
Walk me through `loop()` in C3.	Serial CLI → radio service → handle queued app messages → buttons → battery → app timers → buzzer → OLED check → draw a frame at about 30 fps → `delay(1)`. All non-blocking.

Question	Answer
Why no RTOS tasks in C3 like in v2?	Single core and no audio streaming. One loop is simpler and has no shared state. Only NimBLE has its own task, and it talks to the loop through a queue.
Why a queue between the BLE task and the loop?	Queues are thread-safe and copy data, so there are no shared buffers, no mutex and no race. A full queue drops the line instead of blocking the BLE stack.
Mutex vs semaphore?	A mutex is for exclusive access, has an owner and priority inheritance. A semaphore is for signalling/counting, with no owner.
What is priority inversion?	A high-priority task waits on a lock held by a low-priority task while a medium task runs. Mutex priority inheritance fixes it.
Why a recursive mutex in v2?	The same task could take the UI lock again inside nested drawing calls without deadlocking itself.
What's a watchdog? Why `delay(1)`?	A timer that resets the chip if a task hogs the CPU. `delay(1)` lets the idle task run and feed it.
How do you avoid `delay()`?	Store a timestamp and check `millis() - last >= interval`. Unsigned subtraction stays correct across the 49.7-day wrap.
How is a long BLE message received?	JSON lines end with `\n`. `LineAssembler` joins MTU-sized packets until the newline; oversized lines are dropped whole.
What's in a partition table? Why two app slots?	nvs, otadata, app0, app1, spiffs, coredump. OTA writes the inactive slot, then switches; a failed update never bricks it.
Where are settings stored? How do you protect flash?	NVS namespace `dbc3` via Preferences. Write only when the struct differs from the saved copy (`memcmp`), to limit flash wear.

Question	Answer
What happens in deep sleep? How does it wake?	CPU and most RAM off, only RTC on. BTN1 (GPIO1) pulled low wakes it; it boots from `setup()` again and checks the wake cause.
How do you handle emoji or Hindi text?	UTF-8 is folded to ASCII (é → e); unknown characters are dropped, then word-wrapped. The OLED font is ASCII only.
How does Mochi animate?	Each mood is a set of eye/lid/pupil/mouth parameters; values ease toward targets every frame, with random blinks and glances, and blinks to hide mood swaps.
How do you know the time without WiFi?	The app sends `time` with the UTC epoch and the offset in minutes on every connect. In WiFi mode, NTP.
What's odd about POSIX TZ strings?	The sign is inverted: `UTC-5:30` means UTC+5:30 (India).
What are `#if FEATURE_BATTERY` builds for?	Battery code is compiled only in the battery env, so the USB build never reads a floating GPIO0.
How did you unit-test firmware?	Pure logic in `src/logic/` has no Arduino dependencies, so it compiles with g++ on a PC: 537 checks in CI.

12.4 Wireless and networking

Question	Answer
Explain BLE GAP vs GATT.	GAP: advertising, scanning, connecting (who). GATT: services and characteristics with UUIDs (what data).
What's the Nordic UART Service?	A de-facto standard "serial port over BLE": RX characteristic (phone writes), TX characteristic (device notifies).

Question	Answer
What is MTU and why 247?	Maximum ATT packet size. The default 23 gives 20 data bytes; 247 gives 244 (MTU − 3), so fewer packets per message.
Notify vs indicate?	Notify has no acknowledgement (fast). Indicate is acknowledged (reliable, slower). I used notify; the app reconnects if lost.
What is the CCCD?	Descriptor 0x2902. The client writes it to turn notifications on.
Why is the name in the scan response?	The 31-byte advertising packet is mostly used by the 128-bit service UUID.
HTTP vs WebSocket?	HTTP is request/response, one exchange per request. WebSocket upgrades an HTTP connection to a persistent full-duplex channel, so both sides can push.
MQTT in one sentence?	Publish/subscribe through a broker; clients publish to and subscribe to topics. v1 used HiveMQ Cloud over TLS on 8883.
What was wrong with `setInsecure()`?	It encrypts but skips certificate verification, so a man-in-the-middle could impersonate the broker. Fix: verify the CA certificate.
What is mDNS?	DNS over multicast on the local network (224.0.0.251:5353). It resolves `deskbuddy-c3.local` and advertises `_deskbuddy._tcp` for discovery.
What's a captive portal?	In AP mode, a DNS server answers every name with the device's IP, so the phone opens the setup page automatically.
Why lower TX power to 8.5 dBm?	The Super Mini's antenna distorts at full power, so the AP wasn't visible. Lower power gave a clean signal.
Why disable WiFi modem sleep?	With sleep on, the radio misses multicast packets, so mDNS discovery was unreliable.

Question	Answer
Why is the device the server?	Phones change IPs and Android kills background servers; an outgoing connection from the phone always works.

12.5 Android app

Question	Answer
How do you read other apps' notifications?	A `NotificationListenerService`, enabled by the user under Notification access.
How do you keep the connection alive in the background?	A foreground service (type `connectedDevice`) with an ongoing notification.
How do you avoid spamming duplicates?	Skip ongoing notifications and group summaries, and skip the same key + text within 3 s.
How does the app find the device on WiFi?	NSD (Android's mDNS) discovers `_deskbuddy._tcp` and filters by name for "c3".
What happens when the connection drops?	Reconnect with exponential backoff (BLE 2 s → 30 s max); WebSocket pings every 10 s detect dead links.
Which Bluetooth permissions are needed?	Android 12+: `BLUETOOTH_SCAN`, `BLUETOOTH_CONNECT`. Android ≤ 11: location permission to scan.
Why Jetpack Compose?	Declarative UI; the screens redraw from state, which fits a live mirror of the device.

12.6 Tools, process and behavioural

Question	Answer
Why PlatformIO over the Arduino IDE?	Pinned versions, several build envs (USB, battery, OTA), a command line for CI.

Question	Answer
What does esptool do?	Talks to the ESP's ROM bootloader over serial: sync, write flash at offsets, erase, verify.
What's a merged image?	Bootloader + partition table + boot_app0 + app in one binary, flashed at 0x0 on the C3.
How does your tool make flashing safe?	Verifies SHA-256, refuses non-merged images, keeps NVS unless `--erase`, falls back on baud rates, guides BOOT/RESET.
What does your CI do?	Builds both firmware envs, runs host tests, runs ruff/mypy/pytest on Ubuntu + Windows. A version tag builds and publishes a release with checksums.
How did you use AI responsibly?	AI wrote much of the code; I specified, tested on hardware, debugged, reviewed, and I can explain it. I'm open about it in the README.
Tell me about a failure.	The first C3 firmware had OLED and BLE crashes. I chose to rewrite it from scratch on a newer core, with the lessons documented.
How do you debug embedded problems?	Reproduce with serial logs, isolate with a minimal sketch, check the hardware (multimeter, wiring), change one thing at a time, then add a test or a rule so it doesn't come back.

13. One-page revision sheet

Read this the night before an interview.

All pins, side by side

Signal	v1 (D1 Mini, ESP8266)	v2 (ESP32 DevKit)	C3 (Super Mini)
OLED SDA / SCL	D2 GPIO4 / D1 GPIO5	21 / 22	6 / 7

Signal	v1 (D1 Mini, ESP8266)	v2 (ESP32 DevKit)	C3 (Super Mini)
OLED power	3V3	3V3	3V3
BTN1	D5 GPIO14	32	1 (deep-sleep wake)
BTN2	D7 GPIO13	33	20
BTN3	–	27	–
Touch	–	4 (T0)	–
Red LED	D6 GPIO12	16	3
Yellow LED	D3 GPIO0 (strapping!)	17	4
Green LED	D4 GPIO2 (strapping!)	18	5 (optional, not fitted)
Buzzer	D8 GPIO15 (strapping!)	19 (LEDC)	10 (LEDC)
Speaker	–	DAC 25 (+26) → PAM8403	–
Mic	–	SCK 14, WS 13, SD 34	–
Battery sense	–	–	0 (2 × 220 kΩ divider, battery build)
Keep free	–	0, 2, 5, 12, 15; 6–11	2, 8, 9; 18/19 USB; 21

Numbers

Item	Value
Serial baud	115200
I2C	0x3C; 100 kHz probe, 400 kHz draw (v1/v2 400 kHz)
Frame rate	v1 20 fps (50 ms) · v2/C3 ~30 fps (33 ms) · asleep 10 fps
Debounce / long / power	30 ms / 0.8 s / 3 s

Item	Value
Sleep	C3 120 s · v2 5 min
Notifications	8 kept, popup 4 s; app 15 / title 31 / body 127 chars
State push	on change + every 15 s (C3) · 10 s (v2)
BLE	NUS 6E400001/02/03, MTU 247 → 244 B, TX 3 dBm, name `DeskBuddy-C3`
WiFi	WS :81, HTTP :80, OTA :3232, `deskbuddy-c3.local`, `_deskbuddy._tcp`, TX 8.5 dBm
Portal	`DeskBuddy_C3_Setup` @ 192.168.4.1, 180 s
Battery	divider ÷ 2, empty < 3.35 V for 30 s, LDO via 5V pin
Flash (C3)	nvs 0x9000 · otadata 0xE000 · app0 0x10000 · app1 0x1F0000 (1.875 MB each)
Versions	pioarduino 55.03.36 (core 3.3.6), U8g2 2.36.19, ArduinoJson 7.4.3, NimBLE 2.5.1, WebSockets 2.7.2, WiFiManager 2.0.17
Tests	537 firmware host checks · 257 tool tests (~94 % coverage)

Acronyms

Acronym	Meaning
GPIO	General-purpose input/output
I2C	Inter-Integrated Circuit (2-wire bus)
I2S	Inter-IC Sound (digital audio bus)
UART	Universal asynchronous receiver-transmitter
PWM / LEDC	Pulse-width modulation / ESP32's LED-control PWM peripheral
ADC / DAC	Analog-to-digital / digital-to-analog converter
LDO	Low-dropout regulator

Acronym	Meaning
NVS	Non-volatile storage
OTA	Over-the-air update
RTOS	Real-time operating system
BLE	Bluetooth Low Energy
GAP / GATT / ATT	Generic Access Profile / Generic Attribute Profile / Attribute Protocol
MTU	Maximum transmission unit
CCCD	Client Characteristic Configuration Descriptor
NUS	Nordic UART Service
mDNS / DNS-SD / NSD	Multicast DNS / DNS Service Discovery / Android Network Service Discovery
NTP	Network Time Protocol
MQTT	Message Queuing Telemetry Transport
TLS	Transport Layer Security
AP / STA	Access point / station
CC/CV	Constant current / constant voltage charging
CI	Continuous integration

The five sentences to remember

1. DeskBuddy forwards chosen phone notifications to a desk or pocket OLED companion, so the phone stays away.
2. v1 ESP8266 + MQTT/HTTP → v2 ESP32 + FreeRTOS + own app over WebSocket → C3 ESP32-C3 + BLE/WiFi + battery.
3. C3 runs one non-blocking loop; only the BLE task feeds it complete JSON lines through a FreeRTOS queue.
4. My hardest bugs were a NimBLE callback-ownership crash, an OLED I2C bring-up problem, and an RF issue fixed by lowering TX power.

5. I built a pip-installable flasher with checksum verification, NVS-preserving updates and CI-built releases.